

CFD 2025-26 – Lab 01 – solution

Function spaces and norms:

$$\begin{aligned} L^2(\Omega) &= \left\{ u : \Omega \rightarrow \mathbb{R}^2 \text{ s.t. } \int_{\Omega} |u|^2 d\mathbf{x} < \infty \right\}, \\ H^1(\Omega) &= \left\{ u : \Omega \rightarrow \mathbb{R}^2 \text{ s.t. } \int_{\Omega} (|u|^2 + |\nabla u|^2) d\mathbf{x} < \infty \right\}, \\ \|u\|_{L^2(\Omega)} &= \sqrt{\int_{\Omega} |u|^2 d\mathbf{x}}, \quad \|u\|_{H^1(\Omega)} = \sqrt{\int_{\Omega} (|u|^2 + |\nabla u|^2) d\mathbf{x}}. \end{aligned}$$

Integration by parts formula, valid for any functions $u, v \in H^1(\Omega)$:

$$\int_{\Omega} -(\Delta u) v d\mathbf{x} = \int_{\Omega} \nabla u \cdot \nabla v d\mathbf{x} - \int_{\partial\Omega} (\nabla u \cdot \mathbf{n}) v ds.$$

The full code for this lab is provided in the notebook `CFDlab01.ipynb`.

Exercise 1

- 1 The problem can be rewritten in a more compact form as follows:

$$\begin{cases} -\Delta u + 3u = f, & \text{in } \Omega, \\ \nabla u \cdot \mathbf{n} = d, & \text{on } \partial\Omega, \end{cases}$$

where f and d are proper functions defining the source term and the Neumann boundary conditions. We denote with V the Hilbert space that we will use for the trial and test functions

$$V = H^1(\Omega).$$

Testing the differential equation against a test function $v \in V$ and integrating by parts, the following identity is obtained

$$\begin{aligned} \int_{\Omega} (-\Delta u + 3u) v d\mathbf{x} &= \int_{\Omega} (\nabla u \cdot \nabla v + 3uv) d\mathbf{x} - \int_{\partial\Omega} \nabla u \cdot \mathbf{n} v ds \\ &= \int_{\Omega} \nabla u \cdot \nabla v d\mathbf{x} + \int_{\Omega} 3uv d\mathbf{x} - \int_{\partial\Omega} d v ds, \end{aligned}$$

for all $v \in V$. Then, the weak formulation of the diffusion-reaction problem is given by:

$$\text{find } u \in V \text{ s.t. } \int_{\Omega} \nabla u \cdot \nabla v d\mathbf{x} + \int_{\Omega} 3uv d\mathbf{x} = \int_{\Omega} f v d\mathbf{x} + \int_{\partial\Omega} d v ds, \quad \forall v \in V.$$

Or in abstract form

$$\text{find } u \in V \text{ s.t. } a(u, v) = F(v) \quad \text{for all } v \in V, \quad (1)$$

where

$$a(u, v) := \int_{\Omega} \nabla u \cdot \nabla v d\mathbf{x} + \int_{\Omega} 3uv d\mathbf{x}, \quad F(v) := \int_{\Omega} f v d\mathbf{x} + \int_{\partial\Omega} d v ds.$$

The bilinear form a is trivially continuous and coercive, that is

$$\begin{aligned} \exists M > 0 \text{ s.t. } a(w, v) &\leq M \|w\|_V \|v\|_V \quad \forall v, w \in V, \\ \exists \alpha > 0 \text{ s.t. } a(v, v) &\geq \alpha \|v\|_V^2 \quad \forall v \in V, \end{aligned}$$

since it induces the norm of the space $H^1(\Omega)$ (and thus $M = 3, \alpha = 1$) and the linear functional F is continuous, that is

$$\exists M_F > 0 \text{ s.t. } F(v) \leq M_F \|v\|_V \quad \forall v \in V,$$

(e.g., with $M_F = \max\{\|f\|_{L^2(\Omega)}, C_T \|d\|_{L^2(\partial\Omega)}\}$) therefore Lax-Milgram theorem ensures the well posedness of the problem.

- 2 In order to apply the finite element method, a finite dimensional space $V_h \subset V$ is introduced. Thus, the finite element formulation is given by

$$\text{find } u_h \in V_h \text{ s.t. } a(u_h, v_h) = F(v_h) \quad \text{for all } v_h \in V_h. \quad (2)$$

Let $\{\phi_i\}_{i=1}^{N_{\text{dofs}}}$ be a base for the linear space V_h . Then, the problem can be rewritten in its algebraic form $A\mathbf{x} = \mathbf{b}$, where

$$A_{ij} = a(\phi_j, \phi_i), \quad b_i = F(\phi_i),$$

and the entries x_i are the coefficients related to u_h , i.e.

$$u_h = \sum_{i=1}^{N_{\text{dofs}}} x_i \phi_i.$$

- 3 Generally, in finite element programming the solving algorithm is divided in the following steps:

- 1) mesh generation;
- 2) creation of finite element spaces (distribution of degrees of freedom);
- 3) definition of the problem variational form;
- 4) assembly of matrices and vectors;
- 5) solution of the algebraic problem;
- 6) saving and/or visualization of the output.

We follow exactly the same steps, but be careful that in the first version of our solver, the steps 4) and 5) are merged together. In the solution of the next point we give the details about how to deal with the algebraic formulation. Let's start by defining the domain: in this case it is the unit square $[0, 1]^2$, and the `UnitSquareMesh` of Firedrake is designed for generating exactly such a domain. It accepts (at least) two arguments: the number of subdivisions along x and y axis respectively.

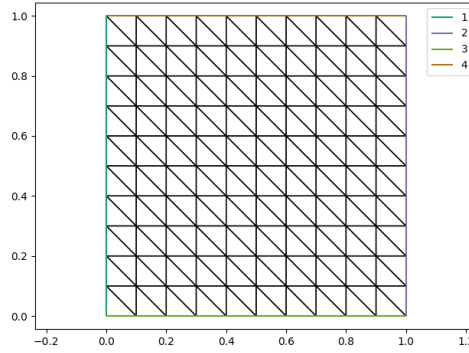


Figure 1: Unit square mesh generated by `UnitSquareMesh` with the option `diagonal='left'`. The colors correspond to the boundary tags.

```
N = 10
mesh = UnitSquareMesh(N, N)
```

The boundary is labeled as shown in Figure 1: the sides are numbered from 1 to 4 in the following order

1. $x = x_{\min}$ (for the unit square, $x_{\min} = 0$);
2. $x = x_{\max}$ (for the unit square, $x_{\max} = 1$);
3. $y = y_{\min}$ (for the unit square, $y_{\min} = 0$);
4. $y = y_{\max}$ (for the unit square, $y_{\max} = 1$).

The mesh can be visualized using the `triplot` function of `matplotlib`¹. Let's now define the finite element space V_h , in this case the well known space $\mathbb{P}^1$ of continuous piecewise linear functions is used:

$$V_h = \{v_h \in H^1(\Omega) : v_h|_K \in \mathbb{P}^1(K) \ \forall K \in \mathbf{T}_h\}$$

```
V = FunctionSpace(mesh, 'P', 1)
```

Let's now define and solve the problem: for this purpose, the trial and test functions u_h and v_h are defined, then the problem reads as follows, when all contributions are explicitly provided:

$$\begin{aligned} \text{find } u_h \in V_h \text{ s.t. } \int_{\Omega} \nabla u_h \cdot \nabla v_h d\mathbf{x} + \int_{\Omega} 3u_h v_h d\mathbf{x} &= \int_{\Omega} 2(y^2 - 1)e^{-x} v_h d\mathbf{x} - \int_{\Gamma_2} y^2 v_h ds \\ &+ \int_{\Gamma_2} y^2 v_h ds + \int_{\Gamma_4} v_h ds, \quad \forall v_h \in V_h. \end{aligned}$$

The corresponding code follows:

¹See <https://matplotlib.org/stable/api/index> for the documentation of this plotting library.

```

u = TrialFunction(V)
v = TestFunction(V)

x = SpatialCoordinate(mesh)
# In Firedrake, analytical functions are yet defined over a
# specific mesh.
f = 2*((x[1]**2)-1)*exp(-x[0])

a = dot(grad(u), grad(v)) * dx + 3*u*v*dx
L = f*v*dx - (x[1]**2)*v*ds(1) + (x[1]**2)*v*ds(2) + v*ds(4)

u_h = Function(V)
solve(a == L, u_h)

```

Then we can plot the solution, using again `matplotlib`:

```

fig, ax = plt.subplots()
q = tripcolor(u_h, axes=ax)
fig.colorbar(q) # show the colorbar

```

In this exercise the algebraic form of the problem is assembled and solved with a single instruction using `solve(a==L, u_h)`, but it is possible to assemble the problem and subsequently solve it, as shown in the next point.

- 4 Now let's see how to assemble matrices and vectors in Firedrake. Having already defined the bilinear form `a` and the right-hand-side functional `L`, we are able to assemble the system matrix `A` and the right-hand-side vector `rhs` simply by

```

A = assemble(a)
rhs = assemble(L)

```

We remark that the right-hand-side vector encodes both the source term and the Neumann boundary conditions. The last step remaining is the computation of the solution to the problem.

```

solver = LinearSolver(A,
                      solver_parameters={'ksp_type': 'preonly',
                                         'pc_type': 'lu'})

u_h_alg = Function(V)
solver.solve(u_h_alg, rhs)

```

The optional `solver_parameters` argument allows to choose which linear solver to use and to set-up its parameters. By default, `LinearSolver` employs the KSP module (<https://petsc.org/main/manual/ksp/>) of the PETSc library (<https://petsc.org>)² This module generally assumes that a preconditioned iterative method is adopted, namely a method that iteratively

²Firedrake could be set up to use also other linear algebra backend libraries. However, in this course we are going to focus on PETSc (or rather Firedrakes' wrappers of PETSc features), which is a very widely employed linear algebra library in many advanced scientific computing libraries.

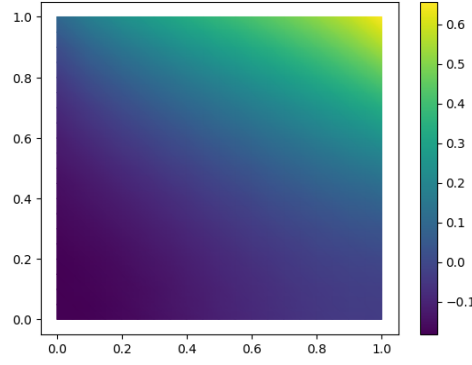


Figure 2: Solution of the diffusion-reaction problem using $N=10$ subdivisions on each axis.

1. applies the preconditioner P , i.e. it solves the system $P\mathbf{z} = \mathbf{r}$ (where the right-hand-side vector is $\mathbf{rhs}$ at the first iteration, and then it depends on the method);
2. performs one or more matrix-vector multiplications using the results of point 1. and of previous iterations.

Therefore, to make `LinearSolver` use a *direct* solver, we actually run only point 1 and only once (`'ksp_type': 'preonly'`), using the LU factorization of the system as a preconditioner P (`'pc_type': 'lu'`):

$$P\mathbf{z} = \mathbf{rhs} \quad \leftrightarrow \quad LU\mathbf{z} = \mathbf{rhs}, \text{ with } LU = A \quad \leftrightarrow \quad (\text{in "Matlab notation"}) \quad \mathbf{z} = U \backslash (L \backslash \mathbf{rhs}).$$

Exercise 2

- 1 In order to keep the notation simpler, the sets $\Gamma_D = \Gamma_1 \cup \Gamma_2$ and $\Gamma_N = \Gamma_3 \cup \Gamma_4$ are introduced, representing the Dirichlet and the Neumann boundary portions, respectively. Then the problem can be rewritten in a more general form

$$\begin{cases} -\Delta u = 0, & \text{in } \Omega, \\ u = g, & \text{on } \Gamma_D, \\ \nabla u \cdot \mathbf{n} = d, & \text{on } \Gamma_N, \end{cases}$$

where g and d are proper functions which define the boundary conditions. We denote with V_0 the Hilbert space

$$V_0 = H_{\Gamma_D}^1(\Omega) = \{v \in H^1(\Omega) : v|_{\Gamma_D} = 0\},$$

and with V_D the space accounting for the Dirichlet data:

$$V_D = \{v \in H^1(\Omega) : v|_{\Gamma_D} = g\}.$$

It is common terminology to refer to V_D as the *trial* space and to V_0 as the *test* space. Testing the differential equation against a test function in V_0 and integrating by parts, the following identity is obtained

$$\int_{\Omega} (-\Delta u) v \, d\mathbf{x} = \int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x} - \int_{\Gamma_D} \nabla u \cdot \mathbf{n} v \, ds - \int_{\Gamma_N} \nabla u \cdot \mathbf{n} v \, ds$$

$$= \int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x} - \int_{\Gamma_N} dv \, ds,$$

for all $v \in V_0$.

We introduce as $\mathcal{R}g \in V_D$ the *lifting* of g , namely a suitable function of V_D such that $\mathcal{R}g|_{\Gamma_D} = g$ and $\tilde{u} = u - \mathcal{R}g \in V_0$. Then, the weak formulation of the Laplace problem can be written as:

$$\begin{aligned} &\text{find } u = \tilde{u} + \mathcal{R}g \in V_D \text{ with } \tilde{u} \in V_0 \text{ s.t.} \\ &\int_{\Omega} \nabla \tilde{u} \cdot \nabla v \, d\mathbf{x} = \int_{\Gamma_N} dv \, ds - \int_{\Omega} \nabla \mathcal{R}g \cdot \nabla v \, d\mathbf{x}, \quad \text{for all } v \in V_0. \end{aligned}$$

Or in abstract form

$$\text{find } u = \tilde{u} + \mathcal{R}g \in V_D \text{ with } \tilde{u} \in V_0 \text{ s.t.} \quad a(\tilde{u}, v) = F(v) - a(\mathcal{R}g, v) \quad \text{for all } v \in V_0,$$

where

$$a(u, v) := \int_{\Omega} \nabla u \cdot \nabla v \, d\mathbf{x}, \quad F(v) := \int_{\Gamma_N} dv \, ds.$$

The bilinear form a is trivially continuous and coercive (thanks to Poincaré inequality), and the linear functional F is continuous, then Lax-Milgram / Babuška-Lax-Milgram theorem ensures the well posedness of the problem.

- 2 In order to apply the finite element method, finite dimensional spaces $V_{0,h} \subset V_0$, $V_{D,h} \subset V_D$ is introduced. Consistently, we introduce a discrete lifting function $\mathcal{R}_h g \in V_{D,h}$. Thus, the finite element formulation is given by

$$\text{find } u_h = \tilde{u}_h + \mathcal{R}_h g \in V_{D,h} \text{ with } \tilde{u}_h \in V_{0,h} \text{ s.t.} \quad a(\tilde{u}_h, v_h) = F(v_h) - a(\mathcal{R}_h g, v_h) \quad \text{for all } v_h \in V_{0,h}.$$

Let be $\{\phi_i\}_{i=1}^{N_{\text{dofs}}}$ a base for the linear space $V_{0,h}$. Then the problem can be rewritten in its algebraic form $A\mathbf{x} = \mathbf{b}$, where

$$A_{ij} = a(\phi_j, \phi_i), \quad \mathbf{b}_i = F(\phi_i) - a(\mathcal{R}_h g, \phi_i),$$

and the entries $\mathbf{x}_i$ are the coefficients related to $\tilde{u}_h$, i.e.

$$\tilde{u}_h = \sum_{i=1}^{N_{\text{dofs}}} \mathbf{x}_i \phi_i.$$

Notice that the lifting consists in a correction of the rhs vector.

- 3 The implementation of this problem is very similar to that in Exercise 1, the only differences being in the definition of the (bi)linear forms

```
a = dot(grad(u), grad(v)) * dx
L = v*ds(3)
```

and the imposition of Dirichlet boundary conditions. In Firedrake, this can be done by creating a `DirichletBC` object for each boundary tag contained in Γ_D , collecting them in a Python `tuple`, and passing this tuple as the `bcs` argument of the `solve` function:

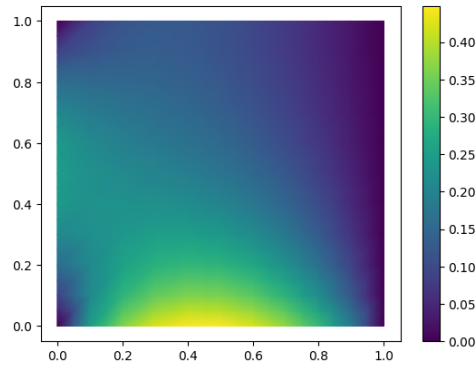


Figure 3: Solution of the Laplace problem using $N = 10$ subdivisions on each axis.

```
bc1 = DirichletBC(V, gD1, 1)
bc2 = DirichletBC(V, gD2, 2)
bcs = (bc1, bc2)
solve(a == L, u_h, bcs=bcs, solver_parameters=params)
```

- 4 We proceed as in Exercise 1, but now we have to account for the essential boundary conditions, too. To see the actual effect of Dirichlet BCs imposition, let us assemble the matrix and r.h.s. both without and with BCs:

```
A = assemble(a)
rhs = assemble(L)
AwithBC = assemble(a, bcs=bcs)
rhswithBC = assemble(L, bcs=bcs)
```

We now want to print the entries of the matrices and rhs: for this we introduce some useful printing options and custom functions, and then we print the results.

```
# print options
np.set_printoptions(precision=1)
np.set_printoptions(linewidth=400)
np.set_printoptions(suppress=True)

# print utility functions

def print_matrix(A):
    Apetsc = A.petscmat
    row_idx = range(0, Apetsc.getSize()[0])
    col_idx = range(0, Apetsc.getSize()[1])
    print(Apetsc.getValues(row_idx, col_idx))
    print() # additional empty line

def print_rhs(b):
    print(b.vector().array())
    print() # additional empty line

print_matrix(A)
```

```
print_rhs(rhs)

print_matrix(AwithBC)
print_rhs(rhswithBC)
```

Comparing the two matrices, we see that some of the rows/columns of $\mathbf{A}$ have been replaced by rows/columns of the identity matrix in $\mathbf{AwithBC}$. In the rhs, instead, some entries have been replaced by 0 in $\mathbf{rhswithBC}$. Indeed, if we denote by $\mathcal{I}_D$ the set of indices corresponding to Dirichlet d.o.f.'s, one can show that

$$\mathbf{AwithBC}_{ij} = \begin{cases} \mathbf{A}_{ij} & \text{if } i, j \notin \mathcal{I}_D \\ \delta_{ij} & \text{if } i \in \mathcal{I}_D \text{ or } j \in \mathcal{I}_D \end{cases} \quad \mathbf{rhswithBC}_i = \begin{cases} \mathbf{rhs}_i & \text{if } i \notin \mathcal{I}_D \\ 0 & \text{if } i \in \mathcal{I}_D \end{cases}$$

The resulting linear system $\mathbf{AwithBC}\mathbf{x} = \mathbf{rhswithBC}$ is thus made of $\dim(V_h) - \#\mathcal{I}_D$ equations corresponding to the original problem tested against the basis functions φ_i for each $i \notin \mathcal{I}_D$, in which only non-Dirichlet entries of the solution are involved, and of $\#\mathcal{I}_D$ equations of the kind $x_i = 0$ for each $i \in \mathcal{I}_D$.

What about the Dirichlet data g and its lifting? This issue will be discussed in the next lab.